THỰC HÀNH CHƯƠNG 2

Xây dựng Backend bằng Vibe Coding

Dự án: Quản lý sinh viên với Node.js, Express và TypeScript

Bạn sẽ dùng AI để xây dựng một Backend nhỏ, từng thành phần một. Trước mỗi lần yêu cầu AI viết code, bạn cần hiểu phần đó dùng để làm gì. Sau khi nhận code, bạn đưa vào dự án, chạy thử và đối chiếu với kết quả yêu cầu.

Hoàn thành tài liệu, bạn sẽ có chương trình xem danh sách sinh viên, tìm sinh viên theo mã và thêm sinh viên mới. Bài thực hành cuối hướng dẫn dùng Postman để kiểm tra những chức năng này.

Cách học trong tài liệu

1. Đọc mục tiêu và lời giải thích trước mỗi bước.
2. Xác định yêu cầu, sau đó mới gửi prompt cho AI.
3. Thực hiện từng thao tác trong VS Code; không chỉ đọc câu trả lời của AI.
4. Kiểm tra kết quả. Nếu chưa đúng, dùng thông tin lỗi để yêu cầu sửa.
5. Trả lời câu hỏi cuối bước bằng lời của bạn.

Vibe Coding trong bài này: mô tả điều muốn xây dựng bằng ngôn ngữ tự nhiên, để AI hỗ trợ tạo hoặc sửa code; bạn chịu trách nhiệm kiểm tra chương trình có làm đúng yêu cầu hay không.

Sản phẩm cần có

- Dự án chạy tại http://localhost:5000.
- Các phần dữ liệu, xử lý và tiếp nhận yêu cầu được tổ chức thành file rõ ràng.
- Một Collection Postman chứa các yêu cầu kiểm thử đã lưu.
- Nhật ký ngắn ghi prompt, kết quả và lỗi bạn đã xử lý.

Dữ liệu tạm nằm trong bộ nhớ và trở về ban đầu khi server khởi động lại. Chưa cần SQL Server, giao diện web hoặc tài khoản đăng nhập để hoàn thành bài này.

Nội dung

Bài 1: Hiểu bài toán và cách làm việc với AI.

Bài 2: Chuẩn bị công cụ và khởi tạo dự án.

Bài 3: Tạo server chạy được.

Bài 4: Thiết kế dữ liệu sinh viên.

Bài 5: Xây dựng Service.

Bài 6: Ghép Controller và Route.

Bài 7: Bổ sung Middleware.

Bài 8: Tìm và thêm sinh viên bằng prompt.

Bài 9: Nhận biết hướng mở rộng Clean Architecture.

Bài 10: Dùng Postman và hiểu ý nghĩa kiểm thử.

Bài 1 Hiểu bài toán trước khi viết prompt

Chúng ta đang xây dựng gì?

Giả sử một ứng dụng của khoa cần xem danh sách sinh viên. Ứng dụng gửi yêu cầu đến Backend. Backend lấy dữ liệu và gửi câu trả lời về. Trong bài này, Postman sẽ đóng vai trò bên gửi yêu cầu thay cho một giao diện web.

Backend là phần chương trình tiếp nhận yêu cầu, xử lý công việc và cung cấp dữ liệu. **API** là cách bên ngoài giao tiếp với chương trình theo các đường dẫn và quy ước xác định.

JSON là một cách biểu diễn dữ liệu bằng văn bản. Ví dụ một sinh viên:

```
{
  "maSV": "SV001",
  "hoTen": "Minh Nhật",
  "email": "minhnhat@example.com",
  "maLop": "25CT114"
}
```

Tên bên trái là tên trường; phần bên phải là giá trị. Chuỗi phải nằm trong dấu nháy kép. JSON không có chú thích hoặc dấu phẩy thừa ở cuối.

Bước 1 Chốt yêu cầu nhỏ đầu tiên

Chức năng đầu tiên chỉ là **xem danh sách sinh viên**. Dữ liệu mẫu gồm SV001 - Minh Nhật và SV002 - Lan Anh. Sau khi chức năng này chạy được, mới thêm tìm theo mã và thêm mới.

Bước 2 Phân biệt nơi thao tác

Nơi thao tác	Bạn làm gì ở đó?
ChatGPT hoặc phần chat của công cụ AI	Gửi prompt, đọc giải thích, lấy code hoặc yêu cầu sửa.
VS Code	Mở dự án, tạo thư mục, tạo file và lưu code.
Terminal trong VS Code	Chạy lệnh cài thư viện, khởi động và kiểm tra dự án.
Postman	Gửi yêu cầu đến Backend và xem câu trả lời.

Bước 3 Chọn một cách làm nhất quán

Tài liệu hướng dẫn theo cách AI trả code và bạn đưa vào VS Code. Nếu dùng công cụ có thể sửa file trực tiếp, vẫn kiểm tra danh sách file thay đổi và tự chạy các bước kiểm tra được nêu trong bài.

Không dán lệnh terminal vào file TypeScript; không dán code TypeScript vào terminal. Luôn yêu cầu AI ghi rõ nội dung thuộc file nào.

Tự kiểm tra

Bạn có thể nói bằng một câu: “Chương trình của tôi nhận yêu cầu gì và trả lại dữ liệu gì?” Nếu chưa trả lời được, hãy đọc lại bài toán trước khi yêu cầu AI viết code.

Bài 1 Cấu trúc một prompt rõ ràng

Một yêu cầu như “làm Backend quản lý sinh viên” chưa cho AI biết phạm vi và cách tổ chức mong muốn. Hãy cung cấp các thông tin dưới đây.

Thành phần	Câu hỏi cần trả lời
ROLE - Vai trò	AI cần hỗ trợ như một người hướng dẫn hay người viết code?
CONTEXT - Bối cảnh	Dự án làm gì, dùng công nghệ nào, đã làm đến đâu?
TASK - Nhiệm vụ	Lần này cần hoàn thành đúng việc gì?
CONSTRAINT - Ràng buộc	Tên file, tên hàm, dữ liệu và giới hạn nào phải giữ?
OUTPUT - Đầu ra	Cần giải thích, code đầy đủ, đường dẫn file và lệnh chạy ra sao?
CHECK - Kiểm tra	Làm sao chứng minh kết quả đúng?

Các nhãn giúp bạn nhớ những thông tin cần nêu; chúng không phải cú pháp bắt buộc của AI. Điều quan trọng là yêu cầu rõ và nhất quán.

Bước 1 Tạo cuộc trò chuyện cho dự án

Mở một cuộc trò chuyện mới, đặt tên dễ nhớ như “Backend quản lý sinh viên”. Gửi prompt thiết lập dưới đây một lần trước các prompt thực hành.

Prompt 0 Thiết lập cách AI hỗ trợ

```
ROLE: Bạn là người hướng dẫn Backend cho sinh viên chưa biết lập trình Backend.
CONTEXT: Tôi xây dựng Backend quản lý sinh viên bằng Node.js, Express 5, TypeScript và pnpm, trên Windows với VS Code. Cấu hình dùng ESM, NodeNext và tsx để chạy lúc phát triển.
TASK: Hỗ trợ tôi từng bước theo nhiệm vụ tôi gửi. Bây giờ chỉ xác nhận cách làm, chưa tạo code.
CONSTRAINT: Dữ liệu SinhVien gồm maSV, hoTen, email, maLop, đều là chuỗi. Cổng 5000. Chưa dùng database, giao diện, JWT hay Docker. Không tự thêm chức năng. Import nội bộ dùng đuôi .js theo cấu hình ESM này.
OUTPUT: Mỗi lần hãy giải thích ngắn vì sao cần làm, liệt kê file tạo/sửa, trả đầy đủ nội dung file cần thay đổi, có chú thích tiếng Việt ở phần quan trọng. Lệnh terminal trình bày riêng. Không dùng dấu ba chấm thay code.
CHECK: Luôn hướng dẫn cách chạy, kết quả mong đợi và một câu hỏi giúp tôi hiểu phần vừa làm. Nếu thiếu thông tin dự án, hỏi hoặc yêu cầu tôi cung cấp file liên quan trước khi sửa.
```

Bước 2 Giữ bối cảnh xuyên suốt

Gửi các prompt tiếp theo trong cùng cuộc trò chuyện. Tuy vậy, AI có thể quên hoặc hiểu sai. Khi sửa file, hãy gửi phiên bản hiện tại của file đó; đừng giả định AI luôn biết code trên máy bạn.

Bài 1 Đọc kết quả AI và xử lý lỗi

Bước 1 Kiểm tra câu trả lời trước khi áp dụng

Đọc danh sách file. Nếu bạn chỉ yêu cầu tạo DTO mà AI dựng cả hệ thống đăng nhập, hãy yêu cầu làm lại đúng phạm vi. Kiểm tra tên trường có giữ maSV, hoTen, email, maLop hay không.

Nếu AI yêu cầu cài thêm thư viện không có trong bài, hỏi thư viện đó giải quyết việc gì. Không tự thêm nhiều thư viện chỉ vì AI đề xuất.

Bước 2 Đưa code vào đúng vị trí

Trong Explorer của VS Code, chuột phải thư mục cần dùng > New File. Đặt đúng tên file AI ghi. Sao chép code bên trong khối code; bỏ các dấu bao như ``` và chữ “typescript”. Lưu bằng Ctrl+S.

Nếu AI trả **toàn bộ file**, thay toàn bộ nội dung file đó. Nếu chưa rõ là đoạn bổ sung hay toàn bộ file, hỏi lại trước khi dán; không nối hai phiên bản server vào cùng một file.

Bước 3 Khi có lỗi, thu thập bằng chứng

Ghi lại: vừa làm bước nào, chạy lệnh gì, mong đợi gì, thực tế ra sao. Sao chép lỗi đầy đủ từ terminal và code file liên quan. Với lỗi API, lưu thêm method, URL, body và response trong Postman.

Prompt sửa lỗi dùng lại cho mọi bài

```
Tôi đang ở bước [ghi tên bước] của dự án quản lý sinh viên.
Tôi vừa thực hiện: [mô tả hoặc lệnh đã chạy].
Kết quả mong đợi: [ghi cụ thể].
Kết quả thực tế/lỗi đầy đủ: [dán nội dung lỗi].
Các file liên quan hiện tại: [dán code hoặc đính kèm file].
Hãy giải thích nguyên nhân bằng ngôn ngữ cho người mới. Nếu chưa đủ thông tin, nói rõ cần thêm gì.
Sau đó đề xuất cách sửa nhỏ nhất, ghi đúng file cần sửa và trả toàn bộ nội dung file đó. Giữ cấu hình
pnpm, ESM, NodeNext, tsx và các chức năng đang hoạt động. Cuối cùng hướng dẫn kiểm tra lại; không
chỉ nói “đã sửa xong”.
```

Bước 4 Xác nhận lỗi thực sự đã được sửa

Chạy lại đúng thao tác từng gây lỗi. Sau đó thử lại chức năng trước đó đang chạy tốt. Việc này giúp phát hiện sửa một chỗ nhưng làm hỏng chỗ khác.

Lưu ý: trước khi thử thay đổi lớn, sao chép phiên bản đang chạy được sang một thư mục dự phòng. Không gửi mật khẩu thật hoặc file chứa thông tin đăng nhập cho AI.

Nhật ký thực hành

Với mỗi chặng, ghi ba dòng: “Tôi yêu cầu AI làm gì?”, “Tôi đã kiểm tra thế nào?”, “Tôi hiểu thêm điều gì?”. Không cần sao chép toàn bộ cuộc trò chuyện vào báo cáo.

Bài 2 Chuẩn bị công cụ

Vì sao cần các công cụ này?

Công cụ	Vai trò trong bài
Node.js	Chạy chương trình JavaScript ở phía Backend.
pnpm	Cài và quản lý thư viện cho dự án; chạy các lệnh dự án đã đặt tên.
Express	Giúp nhận yêu cầu HTTP và gửi phản hồi.
TypeScript	Giúp phát hiện nhiều lỗi về kiểu dữ liệu khi kiểm tra code.
tsx	Chạy file TypeScript lúc phát triển và chạy lại khi lưu file.
VS Code	Nơi tổ chức file và thao tác với code.
Postman	Công cụ gửi yêu cầu kiểm tra API.

Trong bài này, tsx đảm nhiệm việc chạy và tự khởi động lại khi phát triển. TypeScript vẫn được kiểm tra riêng bằng pnpm build; server chạy được chưa chứng minh code hết lỗi kiểu dữ liệu.

Bước 1 Kiểm tra Node.js

Cài Node.js 22 hoặc 24 từ https://nodejs.org/ nếu máy chưa có. Sau khi cài, mở lại VS Code. Chọn Terminal > New Terminal và chạy từng lệnh:

```
node -v
npm -v
```

Đạt khi cả hai lệnh hiện số phiên bản. Nếu không nhận diện lệnh, kiểm tra lại cài đặt Node.js trước khi tiếp tục.

Bước 2 Cài và kiểm tra pnpm

npm được dùng ở bước này để cài pnpm. Trong terminal, chạy:

```
npm install -g pnpm@10
pnpm -v
```

Nếu máy đã có pnpm 10, không cần cài lại. Nếu PowerShell báo chặn npm.ps1 hoặc pnpm.ps1, chọn mũi tên cạnh dấu + ở vùng Terminal > Command Prompt rồi mở terminal mới. Không cần tự thay đổi chính sách bảo mật Windows.

Bước 3 Chuẩn bị nơi hỏi AI

Mở cuộc trò chuyện dự án đã thiết lập bằng Prompt 0. Giữ VS Code và cửa sổ AI cùng mở để đọc prompt và thao tác. Postman được hướng dẫn chi tiết trong bài cuối; hiện tại có thể dùng trình duyệt để thử GET.

Điểm dừng: chỉ tiếp tục khi node -v, npm -v và pnpm -v đều chạy được. Không yêu cầu AI tạo hàng loạt file khi môi trường chưa hoạt động.

Bài 2 Tạo dự án và hiểu các file ban đầu

Bước 1 Tạo thư mục dự án

Trong VS Code, chọn File > Open Folder, chọn một thư mục học tập bạn có quyền ghi. Mở Terminal > New Terminal và chạy lần lượt:

```
mkdir backend-quan-ly-sinh-vien
cd backend-quan-ly-sinh-vien
pnpm init
```

mkdir tạo thư mục, cd chuyển terminal vào thư mục đó, pnpm init tạo thông tin ban đầu của dự án. Sau đó chọn File > Open Folder và mở chính thư mục backend-quan-ly-sinh-vien.

Bước 2 Cài thư viện

Mở terminal trong thư mục dự án, chạy:

```
pnpm add express@5 dotenv@16 cors@2 helmet@8
pnpm add -D typescript@5 tsx@4 @types/node@22 @types/express@5 @types/cors@2
```

Lệnh thứ nhất cài thư viện chương trình dùng: Express, dotenv để đọc cấu hình môi trường, cors để cấu hình việc trình duyệt gọi API khác nguồn, helmet để thiết lập một số HTTP header bảo mật. Postman không thực thi CORS như trình duyệt.

Lệnh thứ hai cài công cụ phát triển. -D lưu vào nhóm devDependencies. Các gói @types/... giúp TypeScript hiểu kiểu dữ liệu của thư viện.

Bước 3 Nhận diện kết quả

Tên	Ý nghĩa
package.json	Ghi thông tin dự án, thư viện và các lệnh chạy.
pnpm-lock.yaml	Ghi phiên bản thư viện đã giải quyết để cài đặt nhất quán.
node_modules	Nơi chứa thư viện được cài. Không tự sửa code ở đây.
src	Thư mục bạn sẽ tạo để chứa mã nguồn của mình.

Prompt 1 Nhờ AI giải thích dự án vừa tạo

```
Tôi đã chạy pnpm init và cài các thư viện của bài. Đây là package.json hiện tại: [dán nội dung file]. Hãy giải thích ngắn name, scripts, dependencies và devDependencies cho người mới. Chỉ giải thích các mục thực sự có trong file; nếu scripts chưa có thì nói rõ. Giải thích vì sao không sửa code trong node_modules. Chưa tạo thêm file ở bước này.
```

Tự kiểm tra: bạn đã mở đúng thư mục nếu nhìn thấy package.json trong Explorer. Khi cài xong, terminal trở lại dấu nhắc nhập lệnh và không có thông báo cài đặt thất bại.

Bài 2 Cây thư mục của dự án

Đọc sơ đồ trước khi tạo file

Đây là cấu trúc khi hoàn thành bài chính. **Thư mục gốc** là backend-quan-ly-sinh-vien: thư mục bạn mở bằng File > Open Folder. Các nhánh thụt vào nằm bên trong thư mục phía trên.

```
backend-quan-ly-sinh-vien/
├── package.json
├── pnpm-lock.yaml
├── tsconfig.json
├── .env
├── .env.example
├── .gitignore
├── src/
│   ├── server.ts
│   ├── dtos/
│   │   └── sinhVien.dto.ts
│   ├── services/
│   │   └── sinhVien.service.ts
│   ├── controllers/
│   │   └── sinhVien.controller.ts
│   ├── routes/
│   │   └── sinhVien.routes.ts
│   ├── middlewares/
│   │   ├── logger.middleware.ts
│   │   └── error.middleware.ts
│   └── utils/
│       └── AppError.ts
├── node_modules/
└── dist/
```

File nào tự sinh, file nào bạn tạo?

pnpm init tạo package.json; bạn sẽ chỉnh cấu hình của file này. pnpm add tạo/cập nhật pnpm-lock.yaml và node_modules. pnpm build tạo dist. Không tự tạo nội dung cho ba phần được sinh tự động này.

Bạn tạo tsconfig.json, .env, .env.example, .gitignore và các file trong src theo từng bài. **Không tạo tất cả file TypeScript rỗng ngay từ đầu.** Chỉ tạo file khi tới bước tương ứng; file rỗng chưa có export có thể khiến import không hoạt động.

Ví dụ đọc đường dẫn

src/controllers/sinhVien.controller.ts nghĩa là: mở thư mục gốc → mở src → mở controllers → chọn file sinhVien.controller.ts. Dấu / trong tài liệu phân tách các cấp thư mục, không phải một phần của tên file.

Bài 2 Bản đồ file và thứ tự tạo

Mỗi dòng dưới đây là một địa chỉ tính từ thư mục gốc. Tạo đúng vị trí để các file có thể tìm thấy nhau.

Đường dẫn từ thư mục gốc	Vai trò và bước thực hiện
package.json	Ghi thư viện và scripts. Sinh ở Bài 2; chỉnh theo mẫu cấu hình trước Bài 3.
tsconfig.json	Cấu hình TypeScript. Bạn tạo ở gốc trước khi build lần đầu.
.env và .env.example	Cấu hình PORT và bản mẫu. Tạo ở gốc trước server trong Bài 3.
.gitignore	Chỉ định phần Git bỏ qua. Tạo ở gốc trong Bài 3.
src/server.ts	Khởi tạo Express, gắn middleware/route, mở cổng. Tạo ở Bài 3.
src/dtos/sinhVien.dto.ts	Khai báo bốn trường sinh viên. Tạo ở Bài 4.
src/services/sinhVien.service.ts	Giữ dữ liệu mẫu và thực hiện chức năng. Tạo ở Bài 5, bổ sung ở Bài 8.
src/controllers/sinhVien.controller.ts	Nhận request, gọi Service, trả response. Tạo ở Bài 6, bổ sung ở Bài 8.
src/routes/sinhVien.routes.ts	Ghép method/đường dẫn với Controller. Tạo ở Bài 6, bổ sung ở Bài 8.
src/middlewares/logger.middleware.ts	Ghi method và đường dẫn. Tạo ở Bài 7.
src/middlewares/error.middleware.ts	Trả lỗi theo quy ước chung. Tạo ở Bài 7, bổ sung AppError ở Bài 8.
src/utils/AppError.ts	Biểu diễn lỗi dự kiến cùng statusCode. Tạo ở Bài 8.

Tạo thư mục và file trong VS Code

1. Ở Explorer, chuột phải vùng trống thuộc thư mục dự án > New Folder > nhập src.
2. Chuột phải src > New Folder > nhập controllers khi đến Bài 6.
3. Chuột phải controllers > New File > nhập sinhVien.controller.ts.
4. Dán code AI trả cho chính file đó, Ctrl+S để lưu.
5. Nhìn các cấp thụt vào trong Explorer hoặc đường dẫn phía trên vùng soạn thảo để kiểm tra vị trí.

Nếu lỡ tạo controllers ngang hàng với src, kéo thư mục vào src, rồi kiểm tra đường dẫn trước khi chạy lại. Chú ý tên AppError.ts viết hoa A và E đúng như sơ đồ.

Bài 2 Cấu hình package.json cụ thể

File nằm ở đâu?

Đường dẫn: backend-quan-ly-sinh-vien/package.json, **ngang hàng với src**. JSON không cho phép chú thích hoặc dấu phẩy thừa. Đây là mẫu hoàn chỉnh cho một dự án mới theo bài:

```
{
  "name": "backend-quan-ly-sinh-vien",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "tsx watch src/server.ts",
    "build": "tsc",
    "start": "node dist/server.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.4.7",
    "express": "^5.1.0",
    "helmet": "^8.1.0"
  },
  "devDependencies": {
    "@types/cors": "^2.8.17",
    "@types/express": "^5.0.1",
    "@types/node": "^22.14.0",
    "tsx": "^4.19.3",
    "typescript": "^5.8.3"
  }
}
```

Áp dụng theo đúng tình trạng dự án

Nếu đã chạy các lệnh cài ở Bài 2, giữ các dependencies/devDependencies đã sinh; đối chiếu và bổ sung private, type và scripts như mẫu. Phiên bản phụ có thể khác do thời điểm cài. Không thay một file đang có chỉ bằng khối scripts.

Nếu tạo dự án mới bằng cách dán nguyên mẫu này, lưu rồi chạy pnpm install tại gốc để cài thư viện và tạo lockfile. Không cần chạy lại pnpm init. Chọn một cách khởi tạo, không làm chồng cả hai.

Ý nghĩa ba lệnh

pnpm dev chạy src/server.ts và theo dõi thay đổi. pnpm build kiểm tra/chuyển code sang dist. pnpm start chạy JavaScript đã build; phải build trước. Trước khi tạo server.ts ở Bài 3, chưa chạy dev hoặc start.

Bài 2 Cấu hình tsconfig.json cụ thể

File nằm ở đâu?

Đường dẫn: backend-quan-ly-sinh-vien/tsconfig.json. Chuột phải thư mục gốc > New File > nhập đúng tên. Dán cấu hình sau và lưu:

```
{
 "compilerOptions": {
  "target": "ES2022",
  "module": "NodeNext",
  "moduleResolution": "NodeNext",
  "rootDir": "./src",
  "outDir": "./dist",
  "strict": true,
  "esModuleInterop": true,
  "skipLibCheck": true,
  "forceConsistentCasingInFileNames": true,
  "noEmitOnError": true
 },
 "include": ["src/**/*.ts"],
 "exclude": ["node_modules", "dist"]
}
```

Cấu hình	Ý nghĩa cần hiểu
target	Chọn mức JavaScript sẽ tạo ra sau khi build.
module và moduleResolution	Dùng cách tổ chức/tìm module phù hợp Node.js với cấu hình ESM của bài.
rootDir / outDir	Đọc mã nguồn trong src, ghi kết quả vào dist.
strict	Bật kiểm tra kiểu chặt chẽ để phát hiện lỗi sớm.
esModuleInterop / skipLibCheck	Hỗ trợ tương thích thư viện và bỏ kiểm tra nội bộ file khai báo kiểu của thư viện.
forceConsistentCasingInFileNames	Giúp phát hiện sử dụng tên file không nhất quán về chữ hoa/thường.
noEmitOnError	Không xuất bản build mới khi có lỗi biên dịch.
include / exclude	Xác định nhóm file TypeScript cần xét và các thư mục bỏ qua.

Kiểm tra đúng lúc

Khi chưa có file TypeScript, build có thể báo chưa tìm thấy đầu vào. Hãy tạo src/server.ts theo Bài 3 trước khi kiểm tra build lần đầu. Sau đó chạy pnpm build; nếu có lỗi, đọc và sửa lỗi thay vì tiếp tục dùng bản dist cũ.

Bài 3 Cấu hình môi trường và file bỏ qua

Tạo .env ở thư mục gốc

Trong Explorer, chuột phải thư mục gốc > New File > nhập .env, gồm dấu chấm ở đầu. Nội dung:

```
PORT=5000
```

Đây là cấu hình cổng, không phải code TypeScript. Không thêm ngoặc nhọn, dấu phẩy hoặc tên file vào bên trong. Không lưu thành .env.txt và không đặt trong src.

Tạo .env.example cũng ở thư mục gốc

Nội dung mẫu của bài cũng là:

```
PORT=5000
```

.env là cấu hình được dùng trên máy; .env.example là mẫu chia sẻ. dotenv mặc định đọc .env khi chương trình được chạy từ gốc dự án, không tự đọc .env.example.

Tạo .gitignore ở thư mục gốc

```
node_modules/
dist/
.env
```

Git sẽ bỏ qua các phần khớp quy tắc này khi chúng chưa được theo dõi. .gitignore không xóa file trên máy và không tự loại file đã được Git theo dõi từ trước. Giữ pnpm-lock.yaml và .env.example để người khác có thể cài/chạy dự án.

Server đọc cấu hình bằng cách nào?

Khi yêu cầu AI tạo src/server.ts, kiểm tra có hai ý tương đương:

```
import 'dotenv/config';
const PORT = Number(process.env.PORT ?? 5000);
```

Đoạn trên chỉ minh họa phần đọc PORT, **không phải toàn bộ server.ts**. Prompt 2 sẽ tạo đầy đủ Express và lệnh lắng nghe. Không nối thêm đoạn này nếu file AI trả đã có cấu hình tương đương.

Quy tắc chạy

Mở terminal tại thư mục có package.json rồi chạy pnpm dev. Nếu đổi PORT, dừng và chạy lại server để chắc chắn .env mới được nạp; cập nhật cả URL trình duyệt và baseUrl trong Postman. Chỉ chạy một server trên cùng một cổng.

Bài 3 Kiểm tra đường dẫn và cấu hình với AI

Hiểu đường dẫn import trước khi sửa

Từ src/controllers/sinhVien.controller.ts, đường dẫn ../services/sinhVien.service.js có nghĩa: đi lên một cấp về src, đi vào services, tìm module sinhVien.service.

- ./ bắt đầu từ thư mục đang chứa file hiện tại.
- ../ đi lên một thư mục cha.
- Với cấu hình ESM/NodeNext của bài, import nội bộ ghi .js dù file bạn đang soạn trong src có đuôi .ts. Bản build trong dist sẽ có file .js tương ứng.

Không đổi file src thành .js chỉ để khớp chữ trong import. Không tự thêm đuôi kép như sinhVien.service.ts.js.

Các đường dẫn cần đối chiếu

Từ file	Đường dẫn import nội bộ điển hình
src/server.ts	./routes/sinhVien.routes.js
src/routes/sinhVien.routes.ts	../controllers/sinhVien.controller.js
src/controllers/sinhVien.controller.ts	../services/sinhVien.service.js
src/services/sinhVien.service.ts	../dtos/sinhVien.dto.js
src/services/sinhVien.service.ts	../utils/AppError.js, chỉ từ Bài 8

Prompt kiểm tra cấu trúc trước khi tiếp tục

Tôi đang ở Bài [ghi số bài], dùng pnpm, Express 5, TypeScript, ESM/NodeNext và tsx.
Đây là cây thư mục hiện tại: [dán danh sách đường dẫn hoặc gửi ảnh Explorer].
Đây là package.json và tsconfig.json: [dán nội dung].
Đây là các file liên quan tới bước hiện tại: [dán nội dung].
Hãy đối chiếu vị trí file, tên export/import, scripts và cấu hình. Chỉ xét những file cần có tới bài hiện tại; không yêu cầu tạo trước các file của bài sau.
Nếu sai, nói rõ file đang nằm đâu, cần chuyển tới đâu, dòng import nào cần sửa. Không tạo dự án mới hoặc đổi công nghệ. Cuối cùng hướng dẫn chạy lệnh từ đúng thư mục và nêu kết quả cần thấy.

Ba mốc đối chiếu

Sau Bài 3: có cấu hình và src/server.ts. Sau Bài 6: có thêm DTO, Service, Controller, Route và GET danh sách chạy được. Sau Bài 8: đủ mọi file src trong cây thư mục, có Middleware và AppError; bắt đầu kiểm thử đầy đủ bằng Postman.

Bài 3 Thiết kế server đầu tiên

Vì sao phải có server?

Server là chương trình chờ nhận yêu cầu. Nếu server chưa chạy, Postman không có chương trình nào để gọi. Chúng ta sẽ tạo một địa chỉ kiểm tra đơn giản trước khi làm chức năng sinh viên.

localhost chỉ máy đang sử dụng. 5000 là cổng chương trình lắng nghe. Đường dẫn / là đường dẫn gốc. Địa chỉ đầy đủ sẽ là http://localhost:5000/.

Bước 1 Yêu cầu AI tạo cấu hình và server

Dán package.json và tsconfig.json hiện tại cùng prompt sau; giữ cấu hình đã thiết lập theo mẫu. Đây là bước tạo chương trình nhỏ chạy được, chưa có các file sinh viên.

Prompt 2 Khởi tạo chương trình chạy được

CONTEXT: Tôi đã tạo dự án và cài thư viện theo bài. Đây là package.json và tsconfig.json hiện tại: [dán hai file].
TASK: Hướng dẫn tạo server Express tối thiểu, chạy ở cổng 5000.
CONSTRAINT:
- Giữ dependencies đã cài. package.json dùng type=module.
- scripts: dev="tsx watch src/server.ts", build="tsc", start="node dist/server.js".
- tsconfig.json giữ đúng mẫu cấu hình đã cung cấp: ES2022, NodeNext, rootDir src, outDir dist, strict, esModuleInterop, skipLibCheck, forceConsistentCasingInFileNames, noEmitOnError đều true; include src/**/*.ts; exclude node_modules và dist.
- src/server.ts dùng import 'dotenv/config', Express, cors, helmet, express.json(). Đọc PORT từ .env, mặc định 5000. Chỉ tạo GET / trả HTTP 200 và JSON {success:true,message:"Backend quản lý sinh viên đang hoạt động"}.
- Chưa import route sinh viên vì các file đó chưa có.
- Tạo .env và .env.example chứa PORT=5000; .gitignore bỏ node_modules, dist và .env.
OUTPUT: Giải thích từng file dùng để làm gì, trả nội dung đầy đủ của các file tạo/sửa. Không xóa các dependencies trong package.json. Nêu thao tác tạo file trong VS Code và lệnh kiểm tra.

Bước 2 Áp dụng đúng các file

Các file .env, .env.example, .gitignore, package.json, tsconfig.json nằm ở gốc dự án. File server.ts nằm trong thư mục src. Không đặt .env vào src.

Hiểu trước khi làm: tsconfig.json hướng dẫn cách kiểm tra và chuyển TypeScript sang JavaScript. .env là nơi đặt cấu hình. .env.example là mẫu cấu hình để người khác biết cần khai báo gì.

Bài 3 Chạy server và kiểm tra lần đầu

Bước 1 Kiểm tra TypeScript

Lưu tất cả file bằng File > Save All. Trong terminal ở gốc dự án, chạy:

```
pnpm build
```

Đạt khi lệnh kết thúc không có lỗi TypeScript và xuất hiện thư mục dist. Đây là code JavaScript được tạo từ src. Bạn sửa file trong src, không sửa trực tiếp file trong dist.

Bước 2 Khởi động server

```
pnpm dev
```

Đạt khi terminal in thông báo lắng nghe tại cổng 5000 theo code AI tạo. Terminal còn chạy và chưa trở lại dấu nhắc là bình thường: server đang chờ yêu cầu.

Bước 3 Mở trình duyệt

Gõ chính xác địa chỉ:

```
http://localhost:5000/
```

Kết quả cần thấy:

```
{
  "success": true,
  "message": "Backend quản lý sinh viên đang hoạt động"
}
```

JSON có thể nằm trên một dòng hoặc nhiều dòng. Điều cần đối chiếu là tên trường và giá trị, không phải cách trình duyệt căn lề.

Bước 4 Quan sát thay đổi

Yêu cầu AI hướng dẫn đổi riêng lời chào, lưu file rồi tải lại trang. Quan sát server được chạy lại. Sau đó khôi phục lời chào chuẩn để giữ kết quả kiểm thử thống nhất.

Ctrl+C dừng server. Muốn chạy lại, nhập pnpm dev. Khi cần chạy pnpm build mà server vẫn mở, dùng dấu + để tạo terminal thứ hai tại cùng thư mục.

Prompt khi chưa chạy được

```
Tôi vừa làm Prompt 2. Lệnh pnpm build có kết quả: [dán]. Lệnh pnpm dev có kết quả: [dán]. Khi mở http://localhost:5000/ tôi thấy: [mô tả]. Đây là package.json, tsconfig.json và server.ts hiện tại: [dán]. Hãy xác định bước đang lỗi, giữ cấu hình ESM/NodeNext/tsx, sửa nhỏ nhất và hướng dẫn tôi thử lại.
```

Tự kiểm tra: nếu tắt server rồi tải lại trang, vì sao không còn nhận được dữ liệu? Bạn phải chạy chương trình nào để trang hoạt động lại?

Bài 4 Thiết kế dữ liệu sinh viên

Vì sao phải thống nhất dữ liệu trước?

Nếu file này dùng hoTen, file khác dùng name, chương trình có thể không tìm thấy thông tin cần đọc. Thống nhất tên trường giúp các phần hiểu cùng một kiểu dữ liệu.

Trường	Kiểu	Ý nghĩa và ví dụ
maSV	string	Mã sinh viên, ví dụ SV001.
hoTen	string	Họ tên, ví dụ Minh Nhật.
email	string	Email liên hệ, ví dụ minhnhat@example.com.
maLop	string	Mã lớp, ví dụ 25CT114.

string nghĩa là chuỗi ký tự. Mã sinh viên dùng chuỗi vì có chữ và có thể chứa số 0 ở đầu phần số. Không dùng thông tin cá nhân thật khi tạo dữ liệu thử.

Bước 1 Hiểu DTO trong phạm vi bài

DTO mô tả cấu trúc dữ liệu trao đổi giữa các phần. File DTO của bài khai báo những trường một sinh viên có. Nó chưa tạo database và chưa lưu danh sách sinh viên.

Prompt 3 Tạo mô tả dữ liệu

```
TASK: Tạo src/dtos/sinhVien.dto.ts cho dự án hiện tại.
CONSTRAINT: Export interface ISinhVienDTO gồm maSV, hoTen, email, maLop; cả bốn đều là string và bắt buộc. Chỉ tạo file này, chưa tạo dữ liệu, Service hoặc database. Không đổi tên các trường.
OUTPUT: Giải thích interface và DTO bằng ví dụ đơn giản. Trả toàn bộ code file, có chú thích tiếng Việt. Nêu nơi tạo file và cách kiểm tra bằng pnpm build. Giải thích vì sao khai báo interface không tự kiểm tra dữ liệu JSON người dùng gửi vào.
```

Bước 2 Tạo file theo phản hồi

Trong VS Code, tạo thư mục dtos bên trong src, rồi tạo sinhVien.dto.ts. Dán code, lưu và chạy pnpm build.

Bước 3 Kiểm tra đúng phạm vi

File phải khai báo đủ bốn trường, không có kết nối SQL. Server gốc vẫn chạy như bài trước; chưa xuất hiện API sinh viên. Việc chưa thấy dữ liệu mới trên trình duyệt ở bước này là bình thường.

Điều cần nhớ: TypeScript hỗ trợ kiểm tra code, nhưng JSON gửi từ Postman vẫn phải được chương trình kiểm tra lúc nhận. Bài thêm sinh viên sẽ làm việc đó.

Tự kiểm tra: nếu thêm trường số điện thoại sau này, vì sao phải thống nhất tên và kiểu trước khi yêu cầu AI sửa nhiều file?

Bài 5 Xây dựng Service lấy danh sách

Service làm gì?

Service thực hiện công việc của chức năng. Trong bước này, công việc là cung cấp danh sách sinh viên. Controller sẽ gọi Service để lấy dữ liệu, sau đó tạo câu trả lời HTTP.

Chưa có database nên Service giữ một mảng dữ liệu mẫu. Mảng là danh sách nhiều phần tử; mỗi phần tử là một sinh viên theo DTO đã thiết kế.

Bước 1 Cung cấp hợp đồng cho AI

Tên hàm được chốt là layDanhSachSinhVien. Các bước sau phải dùng đúng tên này. Dán file DTO hiện có cùng prompt, thay vì chỉ nói "viết Service cho tôi".

Prompt 4 Tạo Service đầu tiên

```
CONTEXT: Dự án đã chạy GET / và có ISinhVienDTO. Đây là DTO hiện tại: [dán file].
TASK: Tạo src/services/sinhVien.service.ts để lấy danh sách sinh viên mẫu.
CONSTRAINT:
- Import type ISinhVienDTO từ ../dtos/sinhVien.dto.js.
- Trong module, lưu mảng sinhVienList gồm đúng hai sinh viên: SV001, Minh Nhật,
minhnhat@example.com, 25CT114; SV002, Lan Anh, lananh@example.com, 25CT114.
- Export hàm layDanhSachSinhVien(): ISinhVienDTO[]. Hàm trả danh sách sinh viên; chưa cần async
hoặc database.
- Service không import Express, không nhận req/res và không tạo HTTP response. Chưa tạo Controller
hoặc Route.
OUTPUT: Giải thích mảng dùng làm gì, hàm trả về gì và ai sẽ gọi hàm. Trả toàn bộ file có chú thích, chỉ rõ
nơi lưu. Hướng dẫn pnpm build và nói rõ chưa có URL sinh viên để mở ở bước này.
```

Bước 2 Tạo file và kiểm tra

Tạo thư mục src/services, tạo file, dán code, lưu. Chạy pnpm build. Nếu lỗi import, đối chiếu tên thư mục và tên file DTO; không tự đổi cả hệ thống sang cấu hình module khác để thử.

Bước 3 Đọc ba điểm quan trọng

Tìm nơi khai báo mảng, nơi khai báo hàm và câu return. Bạn chưa cần nhớ toàn bộ cú pháp, nhưng cần chỉ được dữ liệu nằm ở đâu và hàm nào cung cấp dữ liệu đó.

Checkpoint: build không lỗi, GET / vẫn hoạt động. Chưa có Route gọi Service nên chưa thể mở URL để xem danh sách sinh viên.

Tự kiểm tra: nếu muốn thêm một sinh viên mẫu, bạn sẽ yêu cầu AI sửa file nào? Vì sao không sửa lời chào trong server.ts?

Bài 6 Controller tiếp nhận và phản hồi

Vì sao cần Controller?

Service biết cách lấy dữ liệu. Controller biết cách trả lời yêu cầu HTTP. Tách hai việc giúp bạn biết nên tìm ở đâu khi dữ liệu sai hoặc khi câu trả lời API chưa đúng.

Ở bước này: Controller gọi layDanhSachSinhVien, nhận danh sách, rồi trả JSON gồm success, message và data.

Bước 1 Làm quen ba tên thường gặp

- req viết tắt của request: yêu cầu gửi đến.
- res viết tắt của response: phương tiện tạo câu trả lời.
- next dùng để chuyển tiếp, hoặc chuyển lỗi sang nơi xử lý lỗi.

Chức năng lấy toàn bộ danh sách chưa cần đọc tham số từ req. Các chức năng sau sẽ sử dụng nó.

Prompt 5 Tạo Controller lấy danh sách

```
CONTEXT: Đây là src/services/sinhVien.service.ts hiện tại: [dán file].
TASK: Tạo src/controllers/sinhVien.controller.ts.
CONSTRAINT:
- Export hàm getDanhSachSinhVien nhận req, res, next có kiểu từ Express.
- Gọi đúng layDanhSachSinhVien từ ../services/sinhVien.service.js.
- Thành công: HTTP 200, JSON {success:true,message:"Lấy danh sách sinh viên thành công",data: danh sách nhận từ Service}.
- Có try/catch; khi lỗi thì next(error). Không dùng any tường minh. Sau khi gửi response không tiếp tục gửi lần thứ hai.
- Không tạo lại mảng dữ liệu trong Controller. Chưa sửa server hoặc tạo Route.
OUTPUT: Trả code đầy đủ và giải thích chỗ gọi Service, chỗ đặt status, chỗ trả JSON. Hướng dẫn tạo file và chạy pnpm build. Nhắc rằng bước này chưa có URL mới để gọi.
```

Bước 2 Áp dụng

Tạo src/controllers/sinhVien.controller.ts, dán nội dung và lưu. Chạy pnpm build. Khi build đạt, đối chiếu import của Controller với tên hàm Service thực tế.

Bước 3 Hiểu câu trả lời dự kiến

success: true báo kết quả theo quy ước của ứng dụng. message mô tả kết quả. data chứa danh sách. HTTP 200 là mã trạng thái của phản hồi; nó không phải một trường bắt buộc phải nằm trong JSON.

Tự kiểm tra: Service trả mảng sinh viên; Controller thêm thông tin gì để tạo phản hồi cho người gọi?

Bài 6 Route nối đường dẫn với Controller

Route giải quyết việc gì?

Server có thể nhận nhiều loại yêu cầu. Route chỉ ra yêu cầu ở đường dẫn nào, dùng phương thức nào, sẽ được chuyển đến hàm xử lý nào.

Ta chọn **GET /api/v1/sinh-vien** để lấy danh sách. GET là phương thức dùng để đọc dữ liệu. Route trong file sinh viên dùng /; server gắn thêm tiền tố /api/v1/sinh-vien.

Prompt 6 Ghép chức năng lấy danh sách

```
CONTEXT: Tôi đã có getDanhSachSinhVien trong Controller. Đây là controller và server.ts hiện tại: [dán cả hai file].
TASK: Tạo src/routes/sinhVien.routes.ts và cập nhật src/server.ts để mở API lấy danh sách.
CONSTRAINT:
- File route dùng Router của Express, import getDanhSachSinhVien, khai báo router.get('/', getDanhSachSinhVien), export default router.
- server.ts import router và gắn app.use('/api/v1/sinh-vien', router) sau express.json(), trước app.listen.
- Giữ GET /, cấu hình PORT=5000, dotenv, cors và helmet đang có. Không thêm lại các middleware hoặc app.listen bị trùng.
- Không đưa mảng hoặc nghiệp vụ vào Route. Import nội bộ có đuôi .js.
OUTPUT: Trả đủ hai file và giải thích cách ghép tiền tố với đường dẫn /. Hướng dẫn pnpm build, pnpm dev và URL kiểm tra.
```

Bước 1 Tạo và thay đúng file

Tạo src/routes/sinhVien.routes.ts. Thay server.ts bằng phiên bản đầy đủ đã kiểm tra từ AI. Lưu tất cả và chạy pnpm build.

Bước 2 Chạy và mở URL

Khởi động server nếu chưa chạy. Mở trình duyệt:

```
http://localhost:5000/api/v1/sinh-vien
```

Đạt khi JSON có success=true, thông báo lấy danh sách và data là mảng chứa SV001, SV002. Mở lại / để chắc chắn lời chào vẫn hoạt động.

Bước 3 Tự kể lại luồng

Trình duyệt gửi GET → Route chọn Controller → Controller gọi Service → Service trả danh sách → Controller trả JSON. Dấu mũi tên mô tả luồng xử lý, không phải lệnh bạn phải nhập.

Tự kiểm tra: nếu URL sai thì xem Route; nếu danh sách mẫu sai thì xem Service. Bạn chỉ được hai file đó trong dự án chưa?

Bài 7 Middleware ghi nhận và xử lý lỗi

Vì sao cần Middleware?

Một số việc lặp lại với nhiều yêu cầu, chẳng hạn ghi nhật ký hoặc xử lý lỗi. Middleware giúp đặt chúng tại nơi dùng chung. Logger của bài chỉ ghi method và đường dẫn để bạn quan sát request thực sự đến server.

Middleware ghi log phải gọi next() để request đi tiếp. Middleware xử lý lỗi cần đủ bốn tham số error, req, res, next để Express nhận diện đúng.

Prompt 7 Thêm Middleware đơn giản

```
CONTEXT: Đây là server.ts hiện tại: [dán file]. Controller đang dùng next(error) trong catch.
TASK: Tạo src/middlewares/logger.middleware.ts và src/middlewares/error.middleware.ts, cập nhật server.ts.
CONSTRAINT:
- Logger export loggerMiddleware, in req.method và req.path rồi gọi next(); không ghi body hoặc dữ liệu nhạy cảm.
- Error middleware export errorMiddleware, đủ 4 tham số theo ErrorRequestHandler. JSON body sai cú pháp trả 400 với success=false và message dễ hiểu. Lỗi chưa biết trả 500 với message="Lỗi hệ thống"; chỉ console.error chi tiết ở server.
- Thứ tự server: logger, helmet/cors, express.json(), GET / và route sinh viên, middleware 404 cho đường dẫn không có, cuối cùng errorMiddleware.
- 404 trả {success:false,message:"Không tìm thấy đường dẫn"}. Không thay dữ liệu hoặc đường dẫn đang chạy.
OUTPUT: Trả các file đầy đủ, giải thích vì sao express.json đứng trước Route và middleware lỗi đứng cuối. Nêu cách kiểm tra log và 404.
```

Bước 1 Áp dụng và thử

Tạo hai file middleware, cập nhật server, chạy pnpm build. Gửi lại yêu cầu lấy danh sách. Quan sát terminal có dòng tương tự GET /api/v1/sinh-vien.

Bước 2 Thử đường dẫn không có

Mở http://localhost:5000/khong-co. Đạt khi nhận JSON success=false và thông báo không tìm thấy đường dẫn. Bài Postman sẽ giúp bạn nhìn rõ status 404 của phản hồi này.

Bước 3 Liên hệ với lỗi

Nếu request cứ chờ mãi sau khi thêm logger, kiểm tra middleware có gọi next() hay không. Nếu gửi một request mà terminal lặp log nhiều lần, kiểm tra logger có được gắn trùng hay không.

Tự kiểm tra: tại sao Controller dùng next(error)? Nó chuyển lỗi tới thành phần nào vừa được tạo?

Bài 8 Bổ sung tìm sinh viên theo mã

Thiết kế trước khi yêu cầu AI

Người dùng cần xem một sinh viên cụ thể. Chọn GET /api/v1/sinh-vien/:maSV. Phần :maSV là vị trí thay bằng mã thật, ví dụ /api/v1/sinh-vien/SV001.

Quy ước: tìm thấy trả 200 và data là một object; không tìm thấy trả 404 và success=false. Lúc này 404 diễn tả sinh viên không tồn tại, khác trường hợp không có đường dẫn ở bài trước.

Prompt 8 Mở rộng chức năng đang có

```
CONTEXT: Đây là Service, Controller và Route hiện tại: [dán ba file].
TASK: Thêm tìm sinh viên theo mã, giữ nguyên chức năng lấy danh sách.
CONSTRAINT:
- Service export timSinhVienTheoMa(maSV: string): ISinhVienDTO | undefined, tìm trong mảng đang có; không tạo mảng thứ hai.
- Controller export getSinhVienTheoMa, lấy mã từ req.params.maSV, xử lý kiểu phù hợp Express 5, gọi Service.
- Có dữ liệu: 200, {success:true,message:"Tìm sinh viên thành công",data: sinh viên}. Không có: 404, {success:false,message:"Không tìm thấy sinh viên"}.
- Route thêm GET /:maSV, giữ GET /. Controller có try/catch và next(error).
- Không đổi các tên trường, ESM, cấu hình hoặc dữ liệu mẫu.
OUTPUT: Giải thích tham số URL trước; trả toàn bộ các file cần sửa, chỉ rõ phần mới. Đưa URL thử mã tồn tại và không tồn tại.
```

Bước 1 Áp dụng và build

Đọc lại tên hàm ở ba file để bảo đảm khớp nhau. Dán từng file, lưu và chạy pnpm build. Chỉ thử API khi build không lỗi và server đang chạy.

Bước 2 Thử hai tình huống

- /api/v1/sinh-vien/SV001: có thông tin Minh Nhật.
- /api/v1/sinh-vien/SV999: có thông báo không tìm thấy sinh viên.

Mở lại /api/v1/sinh-vien để xác nhận danh sách vẫn hoạt động. Chức năng cũ phải được thử lại sau mỗi lần mở rộng.

Tự kiểm tra: nếu AI chỉ tạo Route mà quên export hàm trong Controller, chương trình có thể gặp vấn đề ở đâu? Khi báo lỗi cho AI, cần gửi những file nào?

Bài 8 Thiết kế chức năng thêm sinh viên

Vì sao phải chốt quy tắc trước?

“Thêm sinh viên” chưa đủ rõ để AI biết trường nào bắt buộc và khi nào phải từ chối. Trước khi viết prompt, thống nhất yêu cầu dưới đây.

Điều kiện	Hành vi cần có
Có đủ bốn trường, đúng kiểu string	Tiếp tục kiểm tra giá trị.
maSV đúng dạng SV và 3 chữ số	Ví dụ SV003 được chấp nhận.
hoTen, maLop sau khi bỏ khoảng trắng không rỗng	Giá trị rỗng bị từ chối.
email có dạng cơ bản tên@miền.phầnđuôi, không có khoảng trắng	Email rõ ràng sai bị từ chối; chưa kiểm tra hộp thư có thật.
maSV chưa tồn tại	Thêm một sinh viên vào mảng.
maSV đã tồn tại	Không thêm bản ghi thứ hai.

Quy tắc mã SV và email ở đây phục vụ bài thực hành. Chuỗi được bỏ khoảng trắng ở hai đầu trước khi kiểm tra và lưu; mã phải viết hoa đúng dạng, không tự chuyển kiểu số thành chuỗi.

Đường dẫn và kết quả

Dùng **POST /api/v1/sinh-vien**. Dữ liệu gửi kèm đặt trong body dạng JSON. Quy ước phản hồi: thêm được → 201; thiếu hoặc sai dữ liệu → 400; trùng mã → 409. Mọi thất bại phải có success=false và thông báo dễ hiểu.

Bước 1 Chuẩn bị dữ liệu thử

```
{
  "maSV": "SV003",
  "hoTen": "Ngọc Mai",
  "email": "ngocmai@example.com",
  "maLop": "25CT114"
}
```

Gõ URL trong trình duyệt thường gửi GET, không tạo POST có body như ví dụ này. Vì vậy, bài Postman ở cuối sẽ hướng dẫn gửi yêu cầu thêm.

Bước 2 Xác định phần cần sửa

Service kiểm tra dữ liệu và thêm vào mảng. Controller lấy req.body, gọi Service và trả 201. Route khai báo POST. Middleware lỗi chuyển lỗi dự kiến thành status và JSON phù hợp.

Điểm quan trọng: kiểm tra phải diễn ra trước khi ghi vào mảng. Một phản hồi 400 không có ý nghĩa đúng nếu chương trình vẫn thêm dữ liệu lỗi ở phía sau.

Bài 8 Dùng prompt tạo chức năng thêm

Prompt 9 Thêm sinh viên theo quy tắc

CONTEXT: Đây là DTO, Service, Controller, Route và error.middleware.ts hiện tại: [dán các file].
TASK: Thêm POST /api/v1/sinh-vien, giữ nguyên các API GET.
CONSTRAINT:
- Tạo src/utils/AppError.ts: class AppError extends Error, chứa statusCode kiểu number và message; không phụ thuộc Express.
- Service export themSinhVien(input: unknown): ISinhVienDTO. Dữ liệu phải là object, không phải null hoặc array. Bốn trường maSV, hoTen, email, maLop phải là string. Trim các chuỗi. maSV khớp /^SV[0-9]{3}$/; hoTen và maLop không rỗng; email theo kiểm tra cơ bản /^[^\s@]+@[^\s@]+\.[^\s@]+$/.
- Sai dữ liệu: throw AppError(400, thông báo rõ). Sau validation mới kiểm tra mã trùng; trùng thì throw AppError(409, "Mã sinh viên đã tồn tại"). Không sửa mảng khi có lỗi. Chỉ lưu đúng bốn trường DTO, bỏ các trường thừa.
- Hợp lệ: thêm vào mảng hiện có và trả sinh viên mới.
- Controller export taoSinhVien, truyền req.body cho Service, trả 201 với {success:true,message:"Thêm sinh viên thành công",data: sinh viên mới}. Catch chuyển next(error).
- Route thêm POST / gọi taoSinhVien. Error middleware bắt AppError và trả đúng statusCode với {success:false,message}; giữ xử lý JSON sai 400, lỗi chưa biết 500. Không thêm đăng nhập hoặc database.
OUTPUT: Giải thích từng file thay đổi, trả code đầy đủ. Nêu cách build. Liệt kê các ca cần thử ở Postman: hợp lệ, thiếu tên, sai kiểu, email sai, trùng mã. Chưa khẳng định chạy đúng nếu tôi chưa kiểm tra.

Bước 1 Thực hiện

Tạo file AppError trong src/utils; cập nhật các file AI chỉ ra. Lưu và chạy pnpm build. Khi có lỗi, gửi đúng code hiện tại thay vì yêu cầu AI dựng lại từ đầu.

Bước 2 Kiểm tra sơ bộ

GET / và GET danh sách phải vẫn chạy. Việc POST thêm đúng hay sai sẽ được kiểm chứng ở bài Postman; build đạt mới chỉ chứng minh phần kiểm tra TypeScript đã qua.

Tự kiểm tra: nếu người dùng gửi maSV: 3, tại sao không được coi là mã SV003? Nếu chỉ kiểm tra “không rỗng” thì có đủ không?

Bài 9 Nhận biết hướng Clean Architecture

Nhìn lại những gì đã làm

Bạn đã tách Route, Controller và Service. Trong bản hiện tại, Service vừa xử lý nghiệp vụ vừa giữ mảng dữ liệu. Đây là cấu trúc phân tầng đơn giản để bắt đầu, chưa phải một triển khai Clean Architecture đầy đủ.

Giả sử sau này cần lưu bằng SQL Server. Nếu Service biết quá nhiều về cách lưu mảng hoặc câu SQL, việc thay nơi lưu dữ liệu sẽ kéo theo thay đổi trong phần xử lý nghiệp vụ.

Vì sao xuất hiện Repository và Interface?

Repository là phần phụ trách lấy và lưu dữ liệu. **Interface** mô tả các thao tác cần có, ví dụ tìm theo mã hoặc thêm sinh viên. Service làm việc qua hợp đồng đó thay vì tự biết cách SQL Server lưu dữ liệu.

Entity biểu diễn đối tượng nghiệp vụ cốt lõi, ví dụ SinhVien. **Use Case** thể hiện một việc ứng dụng cần thực hiện, ví dụ thêm sinh viên. Các khái niệm này không chỉ là đổi tên thư mục; điều cần quan sát là phần nào phụ thuộc vào phần nào.

Prompt 10 Phân tích trước khi đổi kiến trúc

> Đây là Service và DTO hiện tại của tôi: [dán file]. Tôi muốn hiểu cách tiến tới Clean Architecture, chưa muốn sửa code.
> Hãy chỉ ra đoạn nào là quy tắc nghiệp vụ, đoạn nào là truy xuất dữ liệu. Dùng đúng chức năng thêm sinh viên của tôi để giải thích Entity, Use Case, Interface và Repository cho người mới.
> Đề xuất cách tách nơi lưu mảng sang MemorySinhVienRepository, một interface mô tả list/find/create và cách đưa repository vào Service. Giải thích vì sao Service không nên import Express hoặc mssql.
> Chỉ trình bày trách nhiệm, các file dự kiến và tác động của thay đổi; không tự sinh toàn bộ dự án, không kết nối database.

Cách thực hiện phần mở rộng

Đọc đề xuất AI và đối chiếu với file thật. Chỉ thực hiện refactor sau khi đã kiểm tra bản cơ bản bằng bài Postman. Nếu muốn áp dụng, làm trên bản sao đang chạy được; yêu cầu giữ nguyên URL, cấu trúc JSON và mã trạng thái.

Tự kiểm tra

Bạn có chỉ ra được nơi lưu dữ liệu hiện tại không? Khi đổi sang database, thành phần nào nên chịu trách nhiệm kết nối? Vì sao Controller không cần biết dữ liệu nằm trong mảng hay SQL Server?

Mốc cần đạt của bài chính vẫn là ứng dụng cơ bản và bộ kiểm thử Postman. Không cần hoàn thành phần đổi kiến trúc để bắt đầu bài tiếp theo.

Bài 10 Postman dùng để làm gì?

Vì sao phải kiểm thử API?

AI có thể tạo code chạy được nhưng sai yêu cầu: trả nhầm sinh viên, nhận mã trùng, hoặc báo thành công dù không lưu. Kiểm thử là gửi những yêu cầu có chủ đích rồi so sánh kết quả thực tế với kết quả đã xác định trước.

Một ca kiểm thử gồm: điều kiện ban đầu, dữ liệu gửi, thao tác thực hiện, kết quả mong đợi và kết quả thực tế. Ví dụ: sau khi đã có SV003, gửi thêm SV003 lần nữa; mong đợi 409 và danh sách không tăng.

Postman giúp bạn gửi request mà không cần xây giao diện. Bạn nhìn được cả mã trạng thái lẫn dữ liệu trả về.

Bốn thứ cần biết khi gửi request

Thành phần	Ý nghĩa trong bài
Method	GET để đọc; POST để tạo sinh viên.
URL	Địa chỉ chức năng cần gọi.
Headers	Thông tin mô tả request, ví dụ Content-Type: application/json.
Body	Nội dung gửi kèm, ví dụ thông tin sinh viên mới.

Ba thứ phải đọc trong response

1. **Status:** server báo kết quả loại nào, như 200, 201, 400, 404, 409.
2. **Body:** dữ liệu và thông báo có đúng yêu cầu không?
3. **Tác động:** dữ liệu có thực sự được thêm hoặc được giữ nguyên không? Có thể phải gửi GET tiếp theo để xác nhận.

Test đạt không có nghĩa request luôn nhận 200. Khi cố tình gửi mã trùng, nhận 409 đúng yêu cầu là một ca test đạt. Nếu vẫn nhận 201, ca đó thất bại.

Quy tắc trước khi bắt đầu

Giữ server đang chạy. Để dữ liệu trở về ban đầu, Ctrl+C rồi pnpm dev; danh sách lúc này phải có đúng SV001 và SV002. Không sửa/lưu code giữa các ca kiểm thử liên quan nhau vì chế độ watch có thể khởi động lại và làm mất dữ liệu vừa thêm.

Bài 10 Chuẩn bị Postman và request đầu tiên

Bước 1 Mở Postman trên máy

Tải ứng dụng desktop từ https://www.postman.com/downloads/ nếu chưa cài. Mở ứng dụng và vào workspace làm việc theo giao diện trên máy. Dùng desktop giúp gửi request đến localhost trực tiếp. Nếu dùng bản web, cần Desktop Agent đang chạy và được chọn; Cloud Agent không truy cập localhost trên máy bạn.

Bước 2 Tạo Collection

Trong thanh bên, mở Collections và tạo collection mới bằng nút + hoặc New Collection. Đặt tên **Backend Sinh Vien - Chuong 2**. Collection giống thư mục lưu các request để dùng lại; tên nút có thể khác nhẹ giữa các phiên bản.

Bước 3 Tạo biến địa chỉ

Chọn collection, mở Variables, thêm biến baseUrl với giá trị http://localhost:5000. Lưu nếu giao diện có nút Save. Không đặt dấu / ở cuối giá trị. Nếu phiên bản hiển thị nhiều ô giá trị, bảo đảm giá trị đang dùng tại máy bạn được điền.

Biến giúp đổi cổng hoặc địa chỉ ở một chỗ. Khi dùng {{baseUrl}}, Postman thay bằng giá trị vừa khai báo. Nếu biến bị báo chưa có giá trị, kiểm tra đã lưu request vào đúng collection.

Bước 4 Tạo request kiểm tra server

Chọn Add request trong collection hoặc New > HTTP Request. Đặt tên **01 - Server hoạt động**, chọn method GET và nhập:

```
{{baseUrl}}/
```

Bấm Save để lưu vào collection rồi bấm Send. Không cần Body hoặc Authorization cho API này.

Bước 5 Đọc kết quả

Đạt khi nhận status **200 OK**, body có success: true và thông báo Backend đang hoạt động. Khu vực response nằm bên dưới hoặc bên cạnh tùy bố cục Postman.

Nếu Postman báo Could not send request, chưa có phản hồi HTTP để đánh giá. Kiểm tra terminal server, giá trị biến, cổng và giao thức http trước. Lỗi kết nối này khác với phản hồi 404 hoặc 500 do server gửi.

Tự kiểm tra: nếu đổi PORT thành 5001 và khởi động lại server, bạn cần sửa biến nào trong Postman?

Bài 10 Kiểm thử các chức năng GET

Bước 1 Lấy danh sách

Tạo request **02 - Danh sách sinh viên** trong collection:

```
GET {{baseUrl}}/api/v1/sinh-vien
```

Bấm Send. Kết quả mong đợi: status 200, success=true, data là mảng chứa SV001 và SV002 trên dữ liệu mới khởi động.

Quan sát dấu ngoặc vuông [...]: đó là mảng nhiều sinh viên. Mỗi phần tử là object nằm trong {...}. Nếu server trả lời chào thay vì danh sách, dù status 200, kết quả vẫn sai yêu cầu.

Bước 2 Tìm mã có thật

Tạo request **03 - Tìm SV001**:

```
GET {{baseUrl}}/api/v1/sinh-vien/SV001
```

Mong đợi: status 200; data là một object; data.maSV bằng SV001 và họ tên là Minh Nhật. Không chấp nhận trả toàn bộ danh sách khi yêu cầu chỉ một sinh viên.

Bước 3 Tìm mã không tồn tại

Nhân bản request trên hoặc tạo request **04 - Không có SV999**, thay mã cuối URL thành SV999 và Send.

Mong đợi: status 404; success=false; thông báo không tìm thấy sinh viên. Đây là **kiểm thử trường hợp không thành công** để xác nhận chương trình xử lý tình huống hợp lệ về đường dẫn nhưng không có dữ liệu.

Bước 4 Thử một đường dẫn không có

Tạo request **05 - Đường dẫn không tồn tại**:

```
GET {{baseUrl}}/khong-co
```

Mong đợi: status 404; thông báo không tìm thấy đường dẫn. Cùng status 404 nhưng nguyên nhân khác bước 3. Đọc message giúp xác định đã tới chức năng tìm sinh viên hay chưa.

Ghi kết quả

Với mỗi request, ghi status mong đợi, status thực tế và nhận xét body. Nếu sai, dùng prompt sửa lỗi ở Bài 1 kèm method, URL, response và code liên quan.

Tự kiểm tra: “nhận được dữ liệu” và “nhận đúng dữ liệu được yêu cầu” có giống nhau không?

Bài 10 Gửi POST để thêm sinh viên

Bước 1 Tạo request

Tạo request **06 - Thêm SV003** trong collection. Chuyển method từ GET sang POST. Nhập URL:

```
{{baseUrl}}/api/v1/sinh-vien
```

Bước 2 Chuẩn bị Body

Mở tab Body > chọn raw > chọn JSON trong danh sách định dạng. Dán đúng nội dung dưới đây:

```
{
  "maSV": "SV003",
  "hoTen": "Ngọc Mai",
  "email": "ngocmai@example.com",
  "maLop": "25CT114"
}
```

Trong Headers, kiểm tra có Content-Type: application/json. Khi chọn raw JSON, Postman thường tự thêm header này. Nếu đã có, không thêm bản trùng với giá trị khác.

Bước 3 Gửi và đọc phản hồi

Lưu request rồi Send. Với dữ liệu ban đầu chưa có SV003, mong đợi status **201 Created**, success=true, message thêm thành công và data.maSV=SV003.

Status 201 báo việc tạo mới thành công. Tuy vậy, cần kiểm tra thêm xem dữ liệu thực sự có trong hệ thống hay không.

Bước 4 Xác nhận bằng GET

Gửi lại request lấy danh sách. Mong đợi: có ba sinh viên và xuất hiện SV003. Tạo thêm request **07 - Đọc lại SV003** với GET /api/v1/sinh-vien/SV003; thông tin phải khớp body vừa gửi.

Bước 5 Thử trùng mã ngay sau đó

Không restart server. Gửi lại request **06 - Thêm SV003** với cùng body. Mong đợi **409 Conflict**, success=false, thông báo mã đã tồn tại. Gửi GET danh sách: vẫn chỉ có ba sinh viên.

Lưu request trùng thành **08 - Từ chối trùng SV003** bằng cách nhân bản để tách ca thành công và ca trùng. Hai request giống body nhưng khác điều kiện ban đầu và kết quả mong đợi.

Nếu lần thứ hai vẫn trả 201: chương trình có thể chưa kiểm tra trùng, hoặc đã restart làm mất dữ liệu. Xem terminal trước khi yêu cầu AI sửa.

Bài 10 Kiểm thử dữ liệu không hợp lệ

Bước 1 Kiểm tra thiếu họ tên

Nhân bản request thêm, đặt tên **09 - Thiếu họ tên**. Dùng mã mới SV004 và bỏ trường hoTen:

```
{
 "maSV": "SV004",
 "email": "thunghiem@example.com",
 "maLop": "25CT114"
}
```

Mong đợi: 400 Bad Request, success=false, thông báo giúp biết dữ liệu sai. GET /api/v1/sinh-vien/SV004 phải trả 404. Như vậy mới chứng minh dữ liệu lỗi không bị lưu.

Bước 2 Tạo thêm từng ca độc lập

Mỗi ca dùng lại body đầy đủ của SV004, chỉ thay một chi tiết được chỉ ra. Không để lỗi của ca trước lẫn vào ca sau.

Ca	Thay đổi trong body hợp lệ của SV004	Mong đợi
Sai kiểu mã	maSV đổi thành số 4, không có nháy	400; không thêm
Sai dạng mã	maSV là "ABC004"	400; không thêm
Email sai	email là "khong-phai-email"	400; không thêm
Tên chỉ có khoảng trắng	hoTen là " "	400; không thêm
Thiếu mã lớp	Xóa trường maLop	400; không thêm

Đặt tên request mô tả rõ lỗi đang thử. Sau mỗi ca, kiểm tra danh sách không tăng. Các thông báo lỗi không nhất thiết giống hệt nhau nhưng phải dễ hiểu và đúng nguyên nhân.

Bước 3 Thử JSON sai cú pháp

Tạo request **JSON sai cú pháp**, giữ Body raw JSON và Content-Type. Dán nội dung cố tình thiếu dấu đóng ngoặc:

```
{"maSV":"SV004"
```

Bấm Send. Nếu ứng dụng hiện cảnh báo JSON, đây chính là nội dung sai có chủ đích cần gửi. Mong đợi: 400 và JSON success=false, không phải một trang HTML báo lỗi dài. Ca này kiểm tra middleware đọc JSON và xử lý lỗi, chưa tới bước kiểm tra quy tắc sinh viên.

Ý nghĩa: dữ liệu thiếu trường là JSON đọc được nhưng sai yêu cầu; JSON sai cú pháp là dữ liệu chưa đọc được thành object. Hai lỗi cần được phân biệt khi chẩn đoán.

Bài 10 Viết phép kiểm tra tự động đầu tiên

Vì sao thêm test script?

Kiểm tra bằng mắt phù hợp lúc bắt đầu nhưng dễ bỏ sót khi thử lại nhiều lần. Test script là các điều kiện Postman tự đối chiếu sau khi nhận response. Nó không thay thế việc nghĩ ra ca kiểm thử đúng.

Bước 1 Mở nơi viết test

Mở request **02 - Danh sách sinh viên**. Chọn **Scripts > Post-response**. Ở phiên bản cũ, vị trí tương đương có thể mang tên Tests. Đặt script ở request này, không đặt chung ở Collection vì các request khác có status mong đợi khác nhau.

Bước 2 Thêm đoạn kiểm tra

```
pm.test("Status la 200", function () {
  pm.response.to.have.status(200);
});
pm.test("Tra danh sach co SV001", function () {
  const body = pm.response.json();
  pm.expect(body.success).to.eql(true);
  pm.expect(body.data).to.be.an("array");
  pm.expect(body.data.some(function (sv) {
    return sv.maSV === "SV001";
  })).to.eql(true);
});
```

Lưu rồi Send. Trong vùng response, mở **Test Results**. Mong đợi các phép kiểm tra đều PASS. Nếu response không phải JSON, phép đọc JSON thất bại cũng giúp phát hiện hợp đồng phản hồi chưa đúng.

Bước 3 Hiểu từng phần

pm.test đặt tên cho một phép kiểm tra. pm.response là phản hồi nhận được. pm.expect diễn đạt điều mong đợi. Đoạn trên kiểm tra cả status, kiểu dữ liệu và sự tồn tại của SV001; không chỉ kiểm tra “server có trả lời”.

Bước 4 Quan sát một lần FAIL có chủ đích

Tạm đổi status mong đợi từ 200 thành 201 rồi Send. Test status phải FAIL vì API danh sách trả 200. Sau đó khôi phục 200. Thao tác này chỉ đổi phép kiểm tra ở Postman, không đổi code Backend.

Đừng đổi kỳ vọng đúng thành kết quả sai của chương trình chỉ để có PASS. Khi test thất bại, phải đối chiếu yêu cầu đã chốt để xác định lỗi ở chương trình hay ở phép kiểm tra.

Bài 10 Dùng AI hỗ trợ viết test

Bước 1 Cung cấp kỳ vọng và response thực tế

Muốn AI viết phép kiểm tra có ý nghĩa, cần cho biết request nào, điều kiện dữ liệu, quy tắc đúng và response thực tế. Không chỉ yêu cầu “viết test cho API này”.

Prompt 11 Tạo kiểm tra cho trường hợp trùng mã

ROLE: Hướng dẫn tôi viết test Postman cho người mới.
CONTEXT: Backend quản lý sinh viên lưu mảng trong bộ nhớ. Tôi đã thêm SV003 thành công. Tôi đang dùng request POST /api/v1/sinh-vien để gửi lại cùng mã, server chưa restart.
EXPECTED: Phải trả status 409, success=false và message là chuỗi không rỗng. Không được thêm sinh viên trùng vào danh sách.
ACTUAL: Đây là status và body tôi nhận: [dán response thực tế].
TASK: Trước tiên đối chiếu kết quả thực tế với yêu cầu; nếu sai thì chỉ ra, không thay kỳ vọng cho khớp lỗi. Viết script pm.test đặt ở Scripts > Post-response của request trùng mã để kiểm tra status, success và message.
OUTPUT: Giải thích từng test, nơi dán và cách xem Test Results. Hướng dẫn thêm một request GET để kiểm tra SV003 chỉ xuất hiện một lần; không khẳng định script chỉ đọc response POST đã chứng minh dữ liệu không trùng.

Bước 2 Áp dụng và thử lại

Dán script vào request **08 - Từ chối trùng SV003**. Send và đọc Test Results. Sau đó gửi GET danh sách, đếm số phần tử có maSV bằng SV003; chỉ được một.

Bước 3 Hoàn thành bảng kiểm thử

ID	Điều kiện hoặc request	Kỳ vọng
T01	GET /	200; đúng lời chào
T02	GET danh sách trên seed mới	200; đúng hai sinh viên mẫu
T03	GET SV001	200; đúng Minh Nhật
T04	GET SV999	404; success=false
T05	POST SV003 lần đầu	201; đọc lại được SV003
T06	POST lại SV003, chưa restart	409; SV003 vẫn chỉ có một
T07	POST thiếu tên với mã mới	400; không lưu
T08	POST email sai với mã mới	400; không lưu
T09	POST mã kiểu number	400; không lưu
T10	POST JSON hỏng	400; phản hồi JSON dễ hiểu
T11	GET đường dẫn không có	404; thông báo đường dẫn

Bài 10 Lưu kết quả và nộp bài

Bước 1 Chạy lại theo điều kiện đã định

Restart server để về dữ liệu ban đầu. Chạy T01 đến T11 theo thứ tự, đặc biệt T05 trước T06. Nếu đã thêm SV003 mà không reset, T05 sẽ nhận 409; đây có thể là sai điều kiện chạy test chứ chưa chắc là lỗi chương trình.

Sau bất kỳ ca POST bị từ chối nào, gửi GET để xác nhận dữ liệu không thay đổi. Ghi rõ kiểm tra này trong kết quả thực tế. Không sửa code trong lúc đang chạy chuỗi ca phụ thuộc dữ liệu.

Bước 2 Ghi báo cáo ngắn

Với mỗi ca, ghi: ID, điều kiện ban đầu, method và URL, body nếu có, kết quả mong đợi, kết quả thực tế, Đạt/Chưa đạt. Đính kèm ảnh rõ status và body cho ít nhất một ca thành công và hai ca bị từ chối.

Một ảnh status 200 không đủ chứng minh toàn bộ Backend đúng. Mỗi ca chỉ cung cấp bằng chứng cho yêu cầu và dữ liệu đã thử; chưa chứng minh bảo mật, hiệu năng hoặc mọi tình huống có thể xảy ra.

Bước 3 Lưu và xuất Collection

Lưu tất cả request và scripts. Mở menu dấu ba chấm của Collection > Export nếu phiên bản đang dùng hỗ trợ. Chọn định dạng Collection v2.1 nếu có lựa chọn, lưu file JSON. Nếu vị trí Export khác, tìm trong menu Collection; tránh xuất cả dữ liệu tài khoản cá nhân.

Bước 4 Chuẩn bị bài nộp

- Mã nguồn src, package.json, pnpm-lock.yaml, tsconfig.json, .env.example và .gitignore.
- Collection Postman đã xuất, có tên request dễ hiểu và ít nhất hai request có test script.
- Bảng kết quả T01-T11 cùng ảnh minh chứng.
- Nhật ký prompt: một prompt tạo chức năng, một prompt sửa lỗi nếu có và phần bạn đã điều chỉnh.

Không nộp node_modules, dist hoặc .env thật. Nếu chưa gặp lỗi, ghi “chưa phát sinh lỗi”, không tạo bằng chứng lỗi giả.

Tự giải thích trước khi nộp

1. Route, Controller và Service của chức năng lấy danh sách ở đâu?
2. Vì sao thêm sinh viên phải dùng POST có body?
3. Vì sao nhận 409 có thể là một test đạt?
4. Vì sao phải GET lại sau khi POST?
5. Vì sao restart server làm mất sinh viên vừa thêm?
6. Bạn đã thay đổi gì trong yêu cầu gửi AI, và đã chứng minh thay đổi đó hoạt động ra sao?

Tra cứu nhanh khi thực hành

Hiện tượng	Việc kiểm tra trước
pnpm không được nhận diện	Kiểm tra pnpm -v, cài đặt và mở terminal mới.
Không thấy package.json	Có đang đứng đúng thư mục dự án không?
Cannot find module	Đường dẫn, tên file, export/import và thư viện đã cài.
EADDRINUSE	Cổng đã có server khác. Dừng terminal cũ hoặc đổi PORT và baseUrl đồng bộ.
Could not send request	Server đang chạy chưa; host, cổng và http đúng chưa?
Biến baseUrl báo chưa có giá trị	Request đã thuộc đúng Collection và biến đã được lưu chưa?
req.body chưa có dữ liệu	express.json trước Route; Postman chọn raw JSON và đúng Content-Type.
Request chờ mãi	Middleware có next() hoặc hàm xử lý đã gửi response chưa?
Gửi lần đầu đã nhận 409	Mã đã được thêm trước đó chưa? Có cần reset bộ nhớ không?
Thêm xong rồi không tìm thấy	Watch có restart do lưu file không; POST có thực sự lưu vào mảng chung không?
Test FAIL dù status đúng	Đọc tên phép kiểm tra: body, kiểu dữ liệu hoặc giá trị có thể sai.

Prompt hỏi để hiểu code

> Đây là file [tên file] trong dự án của tôi: [dán code]. Hãy giải thích cho người mới: file nhận gì, gọi file/hàm nào, trả gì và được dùng ở đâu. Chọn tối đa ba đoạn quan trọng để giải thích. Dùng một request cụ thể của chức năng này làm ví dụ. Chưa sửa code.

Nguồn học tập

Tài liệu nền: “Chương 2 Kiến trúc dự án Node.js (MVC / Clean Architecture)”, phần khởi tạo Express với TypeScript, Vibe Coding và bài tập thực hành tại lớp.

Tài liệu tra cứu chính thức:

- Node.js: https://nodejs.org/
- pnpm: https://pnpm.io/installation
- TypeScript: https://www.typescriptlang.org/docs/
- tsx và kiểm tra kiểu: https://tsx.is/faq
- Postman bắt đầu: https://learning.postman.com/docs/getting-started/quick-start
- Postman test scripts: https://learning.postman.com/docs/tests-and-scripts/write-scripts/test-scripts/

Tên nút và bố cục Postman có thể khác giữa các phiên bản; nhiệm vụ vẫn là tạo request, gửi yêu cầu, đọc response và đối chiếu kỳ vọng.